

Question

During the module we've looked at several different ways of implementing a collection of objects, in particular:

- Lists based on arrays
- Linked lists
- Trees, particularly binary search trees
- Hash tables

For each of the following operations, write down the expected time required for the operation. Your answers should assume that the data structure is performing well, i.e., binary search trees are reasonably well balanced and hash tables have a low load factor (i.e., your answer should be the expected time, not the worst case time if those two are different)

(a) *contains*: return true or false depending on whether a particular object is a member of a collection.

(b) *insert*: add a new item to the collection in the appropriate place

Question

(a) Draw the binary search tree that is created if the following numbers are inserted in the tree in the given order.

12 15 3 35 21 42 14

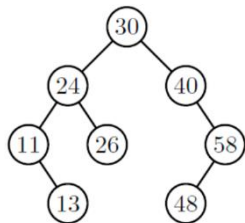
Question

Draw a *balanced* binary search tree containing the same numbers given in part (a).

Question

Consider the Tree below. For each of the algorithms Preorder Tree Walk, In order Tree Walk, and Post order Tree Walk answer the following questions:

(a) Does the algorithm print the keys of elements stored in a binary search tree in a sorted order?

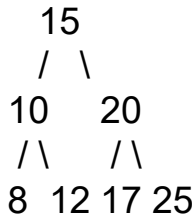


Question

The function **Find(k, R)** shown below is a recursive implementation for searching a key k in a Binary Search Tree (BST) with root node R . Given the following BST:

```
Find( $k, R$ )
```

```
if  $R.\text{Key} = k$ :  
    return  $R$   
else if  $R.\text{Key} > k$ :  
    return Find( $k, R.\text{Left}$ )  
else if  $R.\text{Key} < k$ :  
    return Find( $k, R.\text{Right}$ )
```



1. Trace the execution of **Find(12, R)** step by step, assuming R is the root of the BST shown above. Which nodes are visited, and what is the final output?
2. Trace the execution of **Find(18, R)** for the same tree. Which nodes are visited, and what is the final output?
3. What is the **time complexity** of this algorithm in the **best-case** and **worst-case** scenarios? Provide examples of BSTs that correspond to these cases.
4. Modify the **Find(k, R)** function to handle the case where R is null (i.e., the tree is empty or the key is not found). Provide the updated pseudocode.

Question

Question: Suppose you have an initially empty AVL tree. Insert the following keys in the given order and show the resulting AVL tree after each insertion. Clearly indicate any rotations (if necessary) to maintain the AVL property.

Keys to insert: 10, 20, 30, 15, 25, 5, 35

Instructions:

1. Perform the insertions one at a time.
2. After each insertion, calculate the balance factor for each node.
3. If the tree becomes unbalanced, identify the type of rotation required (LL, RR, LR, or RL) and perform the rotation.
4. Draw the AVL tree after each step, highlighting any rotations performed.